

第五讲 关联规则与Apriori算法

课后作业 · 完整笔记

关联规则应用 · 基本概念 · Apriori算法 · Python实现 · 知识图谱

学生：张椅达 | 学号：202321054055

任务1：关联规则的应用场景

购物篮分析

发现顾客购买行为模式，"啤酒与尿布"经典案例

用途：商品摆放优化·关联推荐·交叉销售

工程故障关联分析

设备A故障→设备B也很可能故障的关联模式

用途：降低维护成本·预防性维修·参《大话数据挖掘》P206

医学规律发现（重点展开）

"症状-基因-药物"潜在关联分析

案例：自闭症(Autism)药物关联网络

19869篇文献·145种药物·构建药物关联网络

发现跨治疗组药物研究越来越普遍(70%+)

金融证券 + 保险 + 餐厅

行业关联选股：发现股票与行业的涨跌联动

保险业：识别高风险关联特征组合以降低风险

餐厅：发现关联大的菜品，辅助生成满意菜单

应用关联规则的两类思维

思维一：发现事物之间的关联，直接解决问题（如行业关联选股法）

思维二：发现关联构建大规模知识图谱，再基于图谱进行更高维度的工作

任务2：关联规则的基本概念

项(Item)与项集 商品/属性，如{网球拍,网球}	事务(Transaction) 每一条购买记录	支持度计数 项集在事务中的出现次数	关联规则 $A \rightarrow B$ 的蕴含关系
支持度(Support) $P(A \cap B)$ ，项集出现频率	置信度(Confidence) $P(B A)$ ，规则的可靠性	提升度(Lift) $Conf/P(B)$ ，>1为正相关	频繁项集/强规则 支持度 $\geq \min_sup$ / 置信度 $\geq \min_conf$

核心公式

$Support(A \rightarrow B) = P(A \cup B)$ | $Confidence(A \rightarrow B) = P(B|A) = P(A \cup B)/P(A)$ | $Lift(A \rightarrow B) = Conf(A \rightarrow B)/P(B)$

关联规则挖掘流程

原始数据 → 设置最小支持度 → 找频繁项集 → 设置最小置信度 → 生成强关联规则 → 结果分析

考试要求：遇到类似J-表6-1的案例，要能将上述概念对号入座

任务2（续）：概念对号入座 — 网球拍示例

事务数据库 D（6条记录）

T1: 网球拍, 网球, 运动鞋
T2: 网球拍, 网球
T3: 网球拍, 羽毛球
T4: 网球拍, 网球, 羽毛球
T5: 运动鞋, 羽毛球
T6: 运动鞋

全项集 $I = \{\text{网球拍}, \text{网球}, \text{运动鞋}, \text{羽毛球}\}$

概念对号入座（考试重点）

概念 → 映射到案例

项(Item): 网球拍、网球、运动鞋、羽毛球

事务(Transaction): T1到T6, 共6条购买记录

项集(Itemset): {网球拍, 网球}是2-项集

支持度计数: {网球拍, 网球}出现了3次 → 支持度计数=3

关联规则: 网球拍→网球 | **支持度**=0.5 | **置信度**=0.6

频繁项集: 设 $\min_sup=0.3$ →支持度 ≥ 0.3 的项集 | **强关联规则:** 同时满足 $\min_conf$

考试要求: 给一个新的案例数据, 能准确找出项、事务、计算支持度和置信度

关联规则：网球拍 → 网球

支持度计算

$\text{Support}(\text{网球拍} \rightarrow \text{网球}) = 3/6 = 0.5$

解释: {网球拍, 网球}在6条记录中出现3次

置信度计算

$\text{Conf}(\text{网球拍} \rightarrow \text{网球}) = 3/5 = 0.6$

解释: 买网球拍的5人中有3人也买了网球

任务3: Apriori算法流程

Apriori算法两大步骤

Step 1: 找频繁项集

- ① 扫描全部数据, 产生候选1-项集 C_1
- ② 根据最小支持度, 由 C_1 筛选出 L_1
- ③ 对 $k>1$: L_k 连接+剪枝 $\rightarrow C_{k+1} \rightarrow L_{k+1}$
- ④ 重复直到 L_k 为空

$C_k \rightarrow L_k \rightarrow C_{k+1} \rightarrow L_{k+1} \rightarrow \dots \rightarrow L_n = \emptyset \rightarrow$ 停止

C_k =候选k-项集 | L_k =频繁k-项集

先验原则: 非频繁项集的超集必非频繁

Step 2: 生成强关联规则

- ⑤ 从每个频繁项集中生成所有可能规则
- ⑥ 计算每条规则的置信度
- ⑦ 保留置信度 $\geq$ 最小置信度的规则

连接操作

L_k 与 L_k 连接: 前(k-1)个项相同 $\rightarrow$ 合并为k+1项集

剪枝操作

删除含非频繁子集的候选 $\rightarrow$ 减少计算量

算法伪代码

```
L1 = {频繁1-项集};
for (k=2; Lk-1≠∅; k++) {
    Ck = apriori_gen(Lk-1); // 连接+剪枝生成候选
    for each 事务 t ∈ D { // 扫描数据库计数
        Ct = subset(Ck, t); // 找出t中包含的候选
        for each 候选 c ∈ Ct : c.count++;
    }
    Lk = {c ∈ Ck | c.count ≥ min_sup}
}
return ∪ Lk (所有频繁项集的并集)
```

任务3（续）：手动计算示例 — 第1-2次扫描

事务数据库（9条记录）

T1: I1, I2, I5

T2: I2, I4

T3: I2, I3

T4: I1, I2, I4

T5: I1, I3

T6: I2, I3

T7: I1, I3

T8: I1, I2, I3, I5

T9: I1, I2, I3

设定最小支持度计数 = 2（即 $\min_sup = 2/9 \approx 22\%$ ）

第1次扫描 $\rightarrow C_1 \rightarrow L_1$

候选1-项集 C_1

I1: 6次 | I2: 7次 | I3: 6次 | I4: 2次 | I5: 2次

频繁1-项集 L_1 （支持度 ≥ 2 ）

$L_1 = \{I1, I2, I3, I4, I5\}$

所有项均满足最小支持度，全部进入 L_1

C_1 大小=5 $\rightarrow L_1$ 大小=5（无剪枝）

第2次扫描 $\rightarrow C_2 \rightarrow L_2$

连接: L_1 自连接生成候选2-项集 C_2

$C_2 = \{I1, I2\}:4, \{I1, I3\}:4, \{I1, I4\}:1, \{I1, I5\}:2, \{I2, I3\}:4, \{I2, I4\}:2, \{I2, I5\}:2, \{I3, I4\}:0, \{I3, I5\}:1, \{I4, I5\}:0$

剪枝（最小支持度 ≥ 2 ） $\rightarrow$ 频繁2-项集 L_2

$L_2 = \{I1, I2\} \{I1, I3\} \{I1, I5\} \{I2, I3\} \{I2, I4\} \{I2, I5\}$

C_2 大小=10, 剪枝后 L_2 大小=6（删除了 $\{I1, I4\}\{I3, I4\}\{I3, I5\}\{I4, I5\}$ ）

删除 $\{I1, I4\}$ 支持度=1 < 2; $\{I3, I4\}\{I4, I5\}=0$; $\{I3, I5\}=1$

任务3（续）：第3-4次扫描 + 生成规则

第3次扫描 $\rightarrow C_3 \rightarrow L_3$

连接操作

$L_2 = \{I1, I2\}\{I1, I3\}\{I1, I5\}\{I2, I3\}\{I2, I4\}\{I2, I5\}$

连接条件：前k-1个项相同

$\{I1, I2\} + \{I1, I3\} \rightarrow \{I1, I2, I3\}$

$\{I1, I2\} + \{I1, I5\} \rightarrow \{I1, I2, I5\}$

经剪枝验证后

$\{I1, I2, I3\}$ 子集都是频繁的 $\rightarrow$ 保留

$\{I1, I2, I5\}$ 子集都是频繁的 $\rightarrow$ 保留

$L_3 = \{I1, I2, I3\} \{I1, I2, I5\}$

Step 2: 生成强关联规则

以频繁3-项集 $\{I1, I2, I5\}$ 为例

可生成6条候选规则，计算每条规则的置信度 ($\text{min_conf}=0.7$)

$I1 \wedge I2 \rightarrow I5$: $\text{conf}=2/4=0.5$ ✗ | $I1 \wedge I5 \rightarrow I2$: $\text{conf}=2/2=1.0$ ✓ | $I2 \wedge I5 \rightarrow I1$: $\text{conf}=2/2=1.0$ ✓

$I1 \rightarrow I2 \wedge I5$: $\text{conf}=2/6=0.33$ ✗ | $I2 \rightarrow I1 \wedge I5$: $\text{conf}=2/7=0.29$ ✗ | $I5 \rightarrow I1 \wedge I2$: $\text{conf}=2/2=1.0$ ✓

强关联规则 ($\text{conf} \geq 0.7$) :

$I1 \wedge I5 \rightarrow I2$ (支持度=2/9, 置信度=1.0) | $I2 \wedge I5 \rightarrow I1$ (支持度=2/9, 置信度=1.0) | $I5 \rightarrow I1 \wedge I2$ (支持度=2/9, 置信度=1.0)

第4次扫描 $\rightarrow C_4 = \Phi$

L_3 自连接生成 C_4 : 前2个项相同

$\{I1, I2, I3\} + \{I1, I2, I5\} \rightarrow \{I1, I2, I3, I5\}$

剪枝：子集 $\{I3, I5\}$ 不是频繁的 $\rightarrow$ 删除

$C_4 = \Phi \rightarrow$ 算法终止

所有频繁项集

L_1 : I1, I2, I3, I4, I5

L_2 : $\{I1, I2\}\{I1, I3\}\{I1, I5\}\{I2, I3\}\{I2, I4\}\{I2, I5\}$

L_3 : $\{I1, I2, I3\}\{I1, I2, I5\}$

任务4：事务数据的矩阵表示

矩阵！矩阵！矩阵！行=事务，列=项，交叉点=元素（0或1）→ 也称为0-1矩阵、布尔矩阵、词向量
这种矩阵表示在自然语言处理（NLP）中也常用，如“词-文档矩阵”

示例：将上述9事务×5项转为矩阵

I1 I2 I3 I4 I5

T1 1 1 0 0 1

T2 0 1 0 1 0

T3 0 1 1 0 0

T4 1 1 0 1 0

T5 1 0 1 0 0

T6 0 1 1 0 0

T7 1 0 1 0 0

T8 1 1 1 0 1

T9 1 1 1 0 0

1 = 事务包含该项 | 0 = 不包含

NLP中的词向量（课堂练习）

句子1：我 / 喜欢 / 数据挖掘

句子2：你 / 喜欢 / 打游戏

构建词-文档矩阵：

我 你 喜欢 数据挖掘 打游戏

句子1 1 0 1 1 0

句子2 0 1 1 0 1

计算句子1与句子2的余弦相似度：

$$\cos(\theta) = (0+0+1+0+0)/(\sqrt{3}\times\sqrt{3}) = 1/3 \approx 0.33$$

任务5: Python实现Apriori (1) — 数据准备

步骤1: 安装并导入依赖库

```
# 安装所需库
pip install mlxtend networkx pandas matplotlib

# 导入库
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
import networkx as nx
import matplotlib.pyplot as plt
```

步骤2: 数据转换与矩阵化

```
# 使用TransactionEncoder将事务数据转换为0-1矩阵
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
trans_df = pd.DataFrame(te_ary, columns=te.columns_)

# 输出转换后的矩阵 (25列 = 25种商品)
print(trans_df.shape) # (100, 25) → 100条交易 × 25种商品
print(trans_df.head()) # 显示前5行0-1矩阵
```

步骤3: 运行结果 (截图展示)

```
[ 加湿器 吸尘器 吹风机 咖啡机 ... 蓝牙耳机 路由器 ]
| 0 False False False False ... False False |
| 1 False False False False ... False False | 100行 × 25列
| 2 False False False False ... False False | 0-1布尔矩阵
| ... ]
```

```
# 读取CSV数据
df = pd.read_csv('products.csv')

# 将每行转为商品列表
transactions = df[['商品1', '商品2', '商品3']].values.tolist()
```

任务5: Python实现 (2) — Apriori挖掘与结果

步骤4: Apriori频繁项集挖掘

```
# 调用Apriori (min_support=0.02 → 至少2次)
frequent_itemsets = apriori(trans_df,
    min_support=0.02,
    use_colnames=True,
    max_len=2)

# 计算实际出现次数
frequent_itemsets['count'] = \
    (frequent_itemsets['support']*len(transactions)).astype(int)

# 过滤出2项集且出现≥2次
qualified = frequent_itemsets[
    (frequent_itemsets['length']==2) &
    (frequent_itemsets['count']>=2)]
```

步骤5: 知识图谱构建与可视化

```
# 创建图对象
G = nx.Graph()

# 添加节点 (所有商品)
G.add_nodes_from(te.columns_)

# 添加边 (关联关系, 权重=共现次数)
for _, row in qualified.iterrows():
    items = list(row['itemsets'])
    G.add_edge(items[0], items[1], weight=row['count'])

# 保存关联关系到CSV
edge_df.to_csv('product_associations.csv', index=False)
```

运行结果

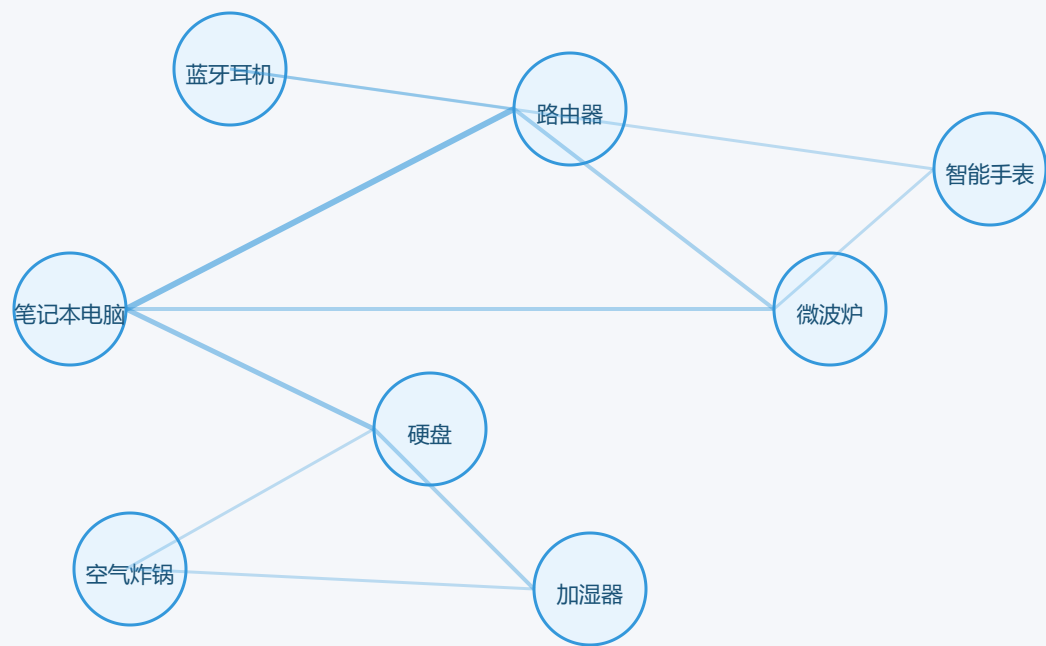
输出 qualified (83条关联关系)

```
support itemsets count
0.02 [加湿器, 咖啡机] 2
0.02 [加湿器, 打印机] 2
0.02 [加湿器, 摄像头] 2
0.03 [智能手表, 加湿器] 3
0.04 [笔记本电脑, 路由器] 4
... ..
```

```
# 可视化
plt.figure(figsize=(14, 10))
pos = nx.spring_layout(G, k=0.5)
nx.draw_networkx_nodes(G, pos, node_size=1500,
    node_color='skyblue')
nx.draw_networkx_labels(G, pos,
    font_size=12, font_color='darkred')
plt.title("商品关联知识图谱")
plt.axis('off')
plt.show()
```

任务5（续）：知识图谱可视化结果 + 关联选股法

商品关联知识图谱



节点：商品 | 边：关联关系 | 边粗：共现强度
83条关联关系 · 25种商品 · 100笔交易

行业关联选股法

背景知识

行业之间存在涨跌联动效应
利用关联规则发现行业间的
共现关系指导选股策略

操作步骤

- ① 理解场景（股票分池）
- ② 分析数据（矩阵行列含义）
- ③ 运行Apriori算法

结果分析与解读

- 挖掘结果发现多个行业间的共现关系
- 当某行业上涨时，关联行业的上涨概率大于先验概率 → 支持关联选股策略
- 关联规则是知识图谱“关系发现”的重要工具

注：注意核对教材中分析过程的错误

第五次作业总结

任务1：展开说明关联规则应用

五大应用场景（购物篮·工程·医学·金融·保险）
重点展开：自闭症药物关联网络分析

任务2：基本概念对号入座

9个概念完整定义 + 网球拍示例（支持度0.5/置信度0.6）
概念到案例的映射表（考试重点）

任务3：Apriori算法流程与计算

两大步骤 + 连接/剪枝 + 伪代码
9事务×5项完整计算 $C_1 \rightarrow L_1 \rightarrow C_2 \rightarrow L_2 \rightarrow C_3 \rightarrow L_3 \rightarrow$ 强规则

任务4：矩阵含义

行=事务·列=项·元素=0/1
NLP词向量对照 + 余弦相似度计算练习

任务5：Python实现Apriori

完整代码展示：数据导入→矩阵转换→apriori挖掘
→ 结果过滤→NetworkX知识图谱构建→可视化
结果：83条关联关系 / 25种商品 / 100笔交易

行业关联选股法

背景知识：行业涨跌联动效应
操作步骤：理解场景→分析数据→运行程序→分析结果
关联规则是知识图谱"关系发现"的重要工具

关联规则本质：从大规模数据中发现隐藏的共现关系 → 知识发现·决策支持